

1.3 넘파이(NumPy)

목차

1. NumPy 개요와 ndarray

1. NumPy란 무엇인가
2. NumPy 모듈 импорт
3. ndarray 생성과 shape
4. ndim - 배열의 차원 확인

2. ndarray의 데이터 타입(dtype) => 저장소 < dtype

같은 타입만 저장
연속 공간 저장

1. 리스트에서 ndarray로 변환과 dtype
2. 서로 다른 타입이 섞였을 때의 형 변환
3. astype()을 이용한 명시적 형 변환

3. ndarray를 편리하게 생성하기 - arange, zeros, ones

4. reshape() - 차원과 크기 변경

1. 기본 사용법
2. 변환 불가능한 경우(오류)
3. -1 인자의 활용
4. 3차원 ↔ 2차원 ↔ 1차원 변환

5. 인덱싱(Indexing)

1. 단일값 추출
2. 슬라이싱(Slicing) 여러 개 추출
3. 팬시 인덱싱(Fancy Indexing)
4. 불린 인덱싱(Boolean Indexing)

핵심은 추출

6. 행렬의 정렬 - sort()와 argsort()

7. 선형대수 연산 - 행렬 내적과 전치 행렬 T

신경망 (딥러닝) → 학습

8. 핵심 요약 및 머신러닝 활용 포인트

1. NumPy 개요와 ndarray

1.1 NumPy란 무엇인가

NumPy(Numerical Python)는 파이썬에서 선형대수 기반의 프로그램을 쉽게 만들 수 있도록 지원하는 대표적인 **수치 계산** 패키지입니다. **루프(loop)**를 사용하지 않고 **대량 데이터의 배열 연산을 가능하게 하므로 매우 빠른 배열 연산 속도를 보장합니다.** **사이킷런(scikit-learn)**을 비롯한 대부분의 **머신러닝/딥러닝 패키지**가 **NumPy 배열을 기본 데이터 구조로 사용**하기 때문에, NumPy를 이해하는 것은 머신러닝 학습의 첫걸음입니다.

■ 머신러닝에서 NumPy가 중요한 이유

- **사이킷런**의 `fit()`, `predict()` 등 대부분의 API가 입력·출력으로 **ndarray**를 사용합니다.
- C 언어 기반 내부 구현으로 파이썬 리스트 대비 **수십~수백 배 빠른** 벡터화 연산을 제공합니다.
- **판다스(Pandas)** DataFrame 역시 내부적으로 ndarray를 기반으로 동작합니다.

numpy
속도가 빠른
저장소

1.2 NumPy 모듈 импорт

```
import numpy as np
```

줄	코드	상세 설명
1	<code>import numpy as np</code> 관례	numpy 모듈을 불러오고 별칭(alias)을 np로 지정합니다. 이후 <code>numpy.array()</code> 대신 <code>np.array()</code> 처럼 짧게 호출할 수 있습니다. np는 전 세계적으로 통용되는 관례적(convention) 별칭이므로 반드시 이 형태를 사용하는 것을 권장합니다.

다차원 배열

1.3 ndarray 생성과 shape

NumPy의 기본 데이터 타입은 **ndarray(N-dimensional array)**입니다. `np.array()` 함수에 파이썬 리스트 등을 인자로 넣으면 ndarray가 생성되며, **shape 속성**으로 **배열의 크기(행과 열의 수)**를 튜플 형태로 확인할 수 있습니다.

```
array1 = np.array([1,2,3])      ndarray object
print('array1 type:', type(array1)) ndarray object
print('array1 array 형태:', array1.shape) (3,)

        ndarray 생성
array2 = np.array([[1,2,3],
                  [2,3,4]])
print('array2 type:', type(array2))
print('array2 array 형태:', array2.shape) (2,3)

        2차원 배열(값이 하나라도)
array3 = np.array([[1,2,3]])
print('array3 type:', type(array3))
print('array3 array 형태:', array3.shape) (1,3)
```

줄	코드	상세 설명
1	<code>array1 = np.array([1,2,3])</code>	1차원 리스트 [1,2,3]을 인자로 전달하여 1차원 ndarray 를 생성합니다. 3개의 데이터를 가진 벡터 형태입니다.

2	<code>print('array1 type:', type(array1))</code>	<code>type()</code> 내장 함수로 객체의 클래스를 확인합니다. 결과는 <code><class 'numpy.ndarray'></code> 로, 파이썬 리스트가 아닌 NumPy 고유의 배열 타입임을 보여줍니다.
3	<code>print('array1 array 형태:', array1.shape)</code>	<code>shape</code> 속성은 배열의 차원별 크기를 튜플로 반환합니다. <code>(3,)</code> 은 "1차원이며 3개의 데이터를 가짐"을 의미합니다. 콤마 뒤가 비어 있는 것이 1차원의 표시입니다.
5~6	<code>array2 = np.array([[1,2,3], [2,3,4]])</code>	리스트 안에 리스트가 2개 들어있는 형태(중첩 리스트)를 전달하여 2행 3열의 2차원 ndarray 를 생성합니다. 머신러닝에서 학습 데이터(피쳐 행렬)가 바로 이 2차원 형태입니다.
7	<code>print('array2 type:', type(array2))</code>	<code>array2</code> 역시 동일한 <code>numpy.ndarray</code> 타입입니다. 차원이 달라도 클래스는 같습니다.
8	<code>print('array2 array 형태:', array2.shape)</code>	<code>(2, 3)</code> 출력 → 로우(행) 2개, 컬럼(열) 3개, 총 $2 \times 3 = 6$ 개의 데이터를 가진 2차원 배열임을 의미합니다.
10	<code>array3 = np.array([[1,2,3]])</code>	데이터 값은 <code>array1</code> 과 동일하지만 대괄호가 두 겹 입니다. 내부에 리스트가 1개 있는 중첩 리스트이므로 1행 3열의 2차원 배열 이 생성됩니다.
11~12	<code>print(... type ... shape ...)</code>	<code>(1, 3)</code> 출력 → 1개의 로우와 3개의 컬럼을 가진 2차원 배열. <code>(3,)</code> 인 <code>array1</code> 과 데이터는 같아도 차원이 다르다 는 점이 핵심입니다.

▶ 실행 결과

```
array1 type: <class 'numpy.ndarray'>
array1 array 형태: (3,)
array2 type: <class 'numpy.ndarray'>
array2 array 형태: (2, 3)
array3 type: <class 'numpy.ndarray'>
array3 array 형태: (1, 3)
```

⚠ 자주 발생하는 실수 - `(3,)`과 `(1,3)`의 차이

`(3,)`은 1차원, `(1,3)`은 2차원입니다. 사이킷런 API는 대부분 2차원 입력을 요구하므로, "Expected 2D array, got 1D array instead"라는 오류를 만나면 `reshape(-1, 1)` 또는 `reshape(1, -1)`로 차원을 맞춰야 합니다. 이 차이를 명확히 이해하는 것이 NumPy 학습의 첫 관문입니다.

차원

1.4 `ndim` - 배열의 차원 확인

```
print('array1: {0}차원, array2: {1}차원, array3: {2}차원'.format(array1.ndim, array2.ndim, array3.ndim))
```

줄	코드	상세 설명
1	<code>'... {0}차원, {1}차원, {2}차원'.format(array1.ndim, array2.ndim, array3.ndim)</code>	<code>ndim</code> 속성은 배열의 차원 수를 정수로 반환합니다. 문자열의 <code>format()</code> 메서드가 <code>{0}</code> , <code>{1}</code> , <code>{2}</code> 자리에 순서대로 각 배열의 <code>ndim</code> 값을 채워 넣습니다. <code>array1</code> 은 1, <code>array2</code> 와 <code>array3</code> 은 2가 출력됩니다.

▶ 실행 결과

```
array1: 1차원, array2: 2차원, array3: 2차원
```

■ 핵심 정리

- **shape** : 차원별 크기를 튜플로 반환 → (3,) , (2, 3)
- **ndim** : 차원의 개수를 정수로 반환 → 1, 2, 3 ...
- **len(shape) == ndim** 관계가 항상 성립합니다.

2. ndarray의 데이터 타입(dtype)

2.1 리스트에서 ndarray로 변환과 dtype

빨리 가능

ndarray 내부의 데이터 타입은 **dtype** 속성으로 확인합니다. 파이썬 리스트와 달리 **ndarray**는 **하나의 배열 안에 한 가지 데이터 타입만 존재**할 수 있습니다.

```
list1 = [1,2,3] ← list
print(type(list1))
array1 = np.array(list1)
print(type(array1))      ndarray      ndarray면 꼭 dtype 써야 함.
print(array1, array1.dtype)
```

줄	코드	상세 설명
1	list1 = [1,2,3]	정수 3개를 가진 순수 파이썬 리스트를 생성합니다.
2	print(type(list1))	<class 'list'> 출력. 아직은 파이썬 기본 자료형입니다.
3	array1 = np.array(list1)	리스트를 np.array()에 전달하여 ndarray로 변환 합니다. 이 과정에서 NumPy가 데이터를 살펴보고 가장 적합한 dtype을 자동으로 결정합니다.
4	print(type(array1))	<class 'numpy.ndarray'> 출력. 변환이 성공했음을 확인합니다.
5	print(array1, array1.dtype)	배열 값 [1 2 3]과 함께 dtype인 int32를 출력합니다(64비트 리눅스 환경에서는 int64로 표시될 수 있음). 리스트 출력과 달리 ndarray는 원소 사이에 콤마가 없이 공백으로 구분되어 표시됩니다.

▶ 실행 결과

```
<class 'list'>
<class 'numpy.ndarray'>
[1 2 3] int32
```

2.2 서로 다른 타입이 섞였을 때의 형 변환

서로 다른 데이터 타입이 섞인 리스트를 ndarray로 변환하면, **더 큰(포괄적인) 데이터 타입으로 일괄 형 변환**이 일어납니다.

```
list2 = [1, 2, 'test']
array2 = np.array(list2)
print(array2, array2.dtype)

list3 = [1, 2, 3.0]
array3 = np.array(list3)
print(array3, array3.dtype)
```

줄	코드	상세 설명
1	<code>list2 = [1, 2, 'test']</code>	정수 2개와 문자열 1개가 섞인 리스트입니다. 파이썬 리스트는 이런 혼합을 허용하지만 ndarray는 허용하지 않습니다.
2	<code>array2 = np.array(list2)</code>	ndarray 변환 시 모든 원소가 문자열(유니코드) 로 통일됩니다. 숫자 1, 2가 문자 '1', '2'로 바뀝니다.
3	<code>print(array2, array2.dtype)</code>	<code>['1' '2' 'test'] <U11</code> 출력. <U11은 "최대 11글자의 유니코드 문자열" 타입을 의미합니다.
5	<code>list3 = [1, 2, 3.0]</code>	정수 2개와 실수(float) 1개가 섞인 리스트입니다.
6	<code>array3 = np.array(list3)</code>	정수보다 실수가 더 포괄적인 타입이므로 모든 원소가 float64 로 통일됩니다. <code>1 → 1.0, 2 → 2.0</code> 으로 변환됩니다.
7	<code>print(array3, array3.dtype)</code>	<code>[1. 2. 3.] float64</code> 출력. 1.은 1.0의 축약 표기입니다.

▶ 실행 결과

```
['1' '2' 'test'] <U11
[1. 2. 3.] float64
```

■ 타입 승격(Type Promotion) 규칙

`int < float < 문자열` 순으로 포괄 범위가 커지며, 혼합 시 항상 더 큰 쪽으로 통일됩니다. 의도치 않은 문자열 변환은 머신러닝 전처리에서 자주 발생하는 버그의 원인이므로 `dtype` 을 습관적으로 확인해야 합니다.

2.3 `astype()`을 이용한 명시적 형 변환

`astype()` 메서드로 원하는 타입을 지정해 변환할 수 있습니다. 대용량 데이터에서 **메모리 절약(float64 → int32 등)**이 필요할 때 주로 사용합니다.

```
ndarray -> dtype = int
array_int = np.array([1, 2, 3])
array_float = array_int.astype('float64')
print(array_float, array_float.dtype)

array_int1 = array_float.astype('int32')
print(array_int1, array_int1.dtype)

array_float1 = np.array([1.1, 2.1, 3.1])
array_int2 = array_float1.astype('int32') 소수점 잘림.
print(array_int2, array_int2.dtype)
```

줄	코드	상세 설명
1	<code>array_int = np.array([1, 2, 3])</code>	정수형(int32) ndarray를 생성합니다.
2	<code>array_float = array_int.astype('float64')</code>	<code>astype()</code> 에 변환할 타입을 문자열 인자('float64') 로 전달합니다. 원본은 변경되지 않고 변환된 새 배열을 반환 합니다.
3	<code>print(array_float, ...)</code>	<code>[1. 2. 3.] float64</code> 출력. 정수 → 실수 변환이 완료되었습니다.
5	<code>array_int1 = array_float.astype('int32')</code>	실수형을 다시 int32로 되돌립니다. 값이 1.0, 2.0, 3.0이므로 손실 없이 1, 2, 3이 됩니다.
6	<code>print(array_int1, ...)</code>	<code>[1 2 3] int32</code> 출력.

8	<code>array_float1 = np.array([1.1, 2.1, 3.1])</code>	소수점 아래 값이 있는 실수 배열을 생성합니다.
9	<code>array_int2 = array_float1.astype('int32')</code>	실수 → 정수 변환 시 소수점 이하는 반올림이 아니라 버림(truncation) 처리됩니다. 1.1 → 1, 2.1 → 2, 3.1 → 3.
10	<code>print(array_int2, ...)</code>	[1 2 3] int32 출력. 데이터 손실이 발생했음에 주의합니다.

▶ 실행 결과

```
[1. 2. 3.] float64
[1 2 3] int32
[1 2 3] int32
```

⚠ 주의

float → int 변환은 소수점 이하를 **무조건 버림**합니다(-1.7 → -1). 반올림이 필요하다면 `np.round( )` 를 먼저 적용한 뒤 `astype( )` 을 호출해야 합니다.

3. ndarray를 편리하게 생성하기 - arange, zeros, ones

범위 지정 0으로 초기화 1로 초기화

테스트용 데이터를 만들거나 배열을 일괄 초기화할 때 사용하는 대표적인 생성 함수 3가지를 살펴봅니다.

3.1 arange() - 연속 값 배열 생성

range와 같음.

차이점: range()는 return list, arange()는 return ndarray

```
sequence_array = np.arange(10)
print(sequence_array)
print(sequence_array.dtype, sequence_array.shape)
           int           (10,)
```

줄	코드	상세 설명
1	<code>sequence_array = np.arange(10)</code>	파이썬 내장 <code>range()</code> 의 배열 버전입니다. 인자 10은 stop 값 이며, 0부터 stop-1인 9까지 연속된 정수로 1차원 배열을 생성합니다. <code>np.arange(start, stop, step)</code> 형태로 시작값·간격도 지정 가능합니다.
2	<code>print(sequence_array)</code>	[0 1 2 3 4 5 6 7 8 9] 출력.
3	<code>print(... dtype, ... shape)</code>	dtype은 int32(환경에 따라 int64), shape는 (10,) → 10개 원소의 1차원 배열입니다.

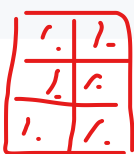
▶ 실행 결과

```
[0 1 2 3 4 5 6 7 8 9]
int32 (10,)
```

3.2 zeros()와 ones() - 0 또는 1로 초기화된 배열

```
zero_array = np.zeros((3,2),dtype='int32')
print(zero_array)
print(zero_array.dtype, zero_array.shape)

one_array = np.ones((3,2)) dtype 안 주면 default는 float
```



```
print(one_array)
print(one_array.dtype, one_array.shape)
```

줄	코드	상세 설명
1	zero_array = np.zeros((3,2), dtype='int32')	첫 번째 인자로 shape 튜플 (3,2)를 전달하여 3행 2열 배열을 만들고 모든 값을 0으로 초기화 합니다. dtype='int32'로 정수형을 명시했습니다.
2~3	print(zero_array) ...	0으로 채워진 3×2 배열, int32 (3, 2)가 출력됩니다.
5	one_array = np.ones((3,2))	모든 값을 1로 초기화 한 3×2 배열을 생성합니다. dtype을 지정하지 않으면 기본값인 float64 가 적용되는 점이 중요합니다.
6~7	print(one_array) ...	1.로 표시되는 실수형 1이 채워진 배열과 float64 (3, 2)가 출력됩니다.

▶ 실행 결과

```
[[0 0]
 [0 0]
 [0 0]]
int32 (3, 2)
[[1. 1.]
 [1. 1.]
 [1. 1.]]
float64 (3, 2)
```

■ 활용 팁

zeros() / ones() 는 결과를 담은 그릇을 미리 만들어 두는 초기화 용도로 자주 쓰입니다. 유사 함수로 초기화 없이 빠르게 공간만 할당하는 np.empty(), 특정 값으로 채우는 np.full() 도 있습니다.

4. reshape() - 차원과 크기 변경

shape 변경

4.1 기본 사용법

reshape() 는 ndarray를 지정된 차원과 크기로 변환합니다. **전체 원소 개수가 유지되어야** 한다는 것이 유일한 제약입니다.

```
array1 = np.arange(10) [0,1,2,3,4,5,6,7,8,9] 1차원 배열, (10,)
print('array1:\n', array1)
array2 = array1.reshape(2,5) 2행 5열
print('array2:\n', array2)
array3 = array1.reshape(5,2) 5행 2열
print('array3:\n', array3)
```

0	1	2	3	4
5	6	7	8	9

0	1
2	3
4	5
6	7
8	9

reshape(3,3) 9 <= error
두 숫자 곱이 10이어야.

줄	코드	상세 설명
1	array1 = np.arange(10)	0~9까지 10개 원소를 가진 1차원 배열을 생성합니다.
2	print('array1:\n', array1)	\n(줄바꿈 문자)을 넣어 레이블 다음 줄에 배열을 출력합니다.
4	array2 = array1.reshape(2,5)	10개 원소를 2행 5열 의 2차원 배열로 변환합니다. 원소는 행 우선(row-major) 순서로 채워집니다: 첫 행에 0~4, 둘째 행에 5~9.

7	<code>array3 = array1.reshape(5,2)</code>	같은 10개 원소를 5행 2열 로 변환합니다. $2 \times 5 = 10$, $5 \times 2 = 10$ 으로 원소 수가 같아 모두 가능합니다.
---	---	--

▶ 실행 결과

```
array1:
[0 1 2 3 4 5 6 7 8 9]
array2:
[[0 1 2 3 4]
 [5 6 7 8 9]]
array3:
[[0 1]
 [2 3]
 [4 5]
 [6 7]
 [8 9]]
```

4.2 변환 불가능한 경우 - ValueError 발생

```
array1.reshape(4,3)
```

줄	코드	상세 설명
1	<code>array1.reshape(4,3)</code>	$4 \times 3 = 12$ 인데 array1의 원소는 10개이므로 크기가 맞지 않아 ValueError가 발생합니다. reshape는 원소를 늘리거나 줄이지 못하며, 오직 "재배치"만 가능합니다.

▶ 실행 결과 (오류)

```
ValueError: cannot reshape array of size 10 into shape (4,3)
```

4.3 -1 인자의 활용 - 자동 크기 계산

인자에 **-1**을 사용하면 "다른 축의 크기에 맞춰 **알아서 계산하라**"는 의미가 됩니다. 실무에서 매우 자주 사용되는 필수 기법입니다.

```
array1 = np.arange(10)
print(array1)

# 행열 열은 무조건 5개. -1 주면 계산 안 하고 10/5 해서 2 줌.
array2 = array1.reshape(-1,5) # (2,5) 2는 자동계산(-1 자리)
print('array2 shape:', array2.shape)

array3 = array1.reshape(5,-1)
print('array3 shape:', array3.shape)
```

줄	코드	상세 설명
1~2	<code>array1 = np.arange(10)</code>	0~9의 1차원 배열을 생성 후 출력합니다.
4	<code>array2 = array1.reshape(-1,5)</code>	"열은 5개로 고정, 행은 -1로 자동 계산"을 지시합니다. 전체 10개 ÷ 5열 = 2행 이 자동으로 결정되어 (2, 5)가 됩니다.
7	<code>array3 = array1.reshape(5,-1)</code>	"행은 5개로 고정, 열은 자동 계산" → $10 \div 5 = \mathbf{2열}$, 즉 (5, 2)가 됩니다.

▶ 실행 결과

```
[0 1 2 3 4 5 6 7 8 9]
array2 shape: (2, 5)
array3 shape: (5, 2)
```

```
array1 = np.arange(10)
array4 = array1.reshape(-1,4)
```

줄	코드	상세 설명
2	array4 = array1.reshape(-1,4)	-1을 쓰더라도 10이 4로 나누어떨어지지 않으므로 자동 계산이 불가능하여 ValueError가 발생합니다. -1은 만능이 아니라 "나눗셈이 성립할 때"만 동작합니다.

▶ 실행 결과 (오류)

```
ValueError: cannot reshape array of size 10 into shape (4)
```

4.4 3차원 ↔ 2차원 ↔ 1차원 변환

`reshape(-1, 1)` 은 어떤 형태의 배열이든 **"N행 1열의 2차원 배열"**로 통일해 주는 관용 표현입니다. 사이킷런에 단일 피처를 입력할 때 필수적으로 사용됩니다.

```
array1 = np.arange(8)          [0,1,2,3,4,5,6,7]
array3d = array1.reshape((2,2,2)) 3차원 배열
print('array3d:\n',array3d.tolist())

# 3차원 ndarray를 2차원 ndarray로 변환
array5 = array3d.reshape(-1,1) 2차원으로 변경
print('array5:\n',array5.tolist())
print('array5 shape:',array5.shape)

# 1차원 ndarray를 2차원 ndarray로 변환
array6 = array1.reshape(-1,1) 1차원 -> 2차원
print('array6:\n',array6.tolist())
print('array6 shape:',array6.shape)
```

줄	코드	상세 설명
1	array1 = np.arange(8)	0~7까지 8개 원소의 1차원 배열을 생성합니다.
2	array3d = array1.reshape((2,2,2))	shape 튜플 (2,2,2)를 전달해 3차원 배열 로 변환합니다. $2 \times 2 \times 2 = 8$ 로 원소 수가 일치합니다. "2개의 면, 각 면은 2행 2열" 구조입니다.
3	print(... array3d.tolist())	<code>tolist()</code> 는 ndarray를 파이썬 리스트로 변환 합니다. 중첩 구조가 콤마로 구분되어 차원 구조를 눈으로 확인하기 쉬워집니다.
6	array5 = array3d.reshape(-1,1)	3차원 배열을 "열 1개, 행은 자동"인 2차원(8행 1열) 으로 변환합니다. 몇 차원이든 <code>reshape(-1, 1)</code> 한 번으로 통일됩니다.
7~8	print(... tolist / shape)	중첩 리스트 형태와 (8, 1) shape를 출력합니다.
11	array6 = array1.reshape(-1,1)	1차원 배열도 동일하게 8행 1열의 2차원 으로 변환됩니다.
12~13	print(... tolist / shape)	array5와 동일한 (8, 1) 결과를 확인합니다.

▶ 실행 결과

```
array3d:
[[[0, 1], [2, 3]], [[4, 5], [6, 7]]]
array5:
[[0], [1], [2], [3], [4], [5], [6], [7]]
array5 shape: (8, 1)
array6:
[[0], [1], [2], [3], [4], [5], [6], [7]]
array6 shape: (8, 1)
```

■ 머신러닝 실무 포인트

X: Data, y: 답
사이킷런의 `fit(X, y)` 에서 **X는 반드시 2차원이어야** 합니다. 피처가 하나뿐인 1차원 데이터는 `X.reshape(-1, 1)` 로 변환하는 것이 표준 패턴입니다. 반대로 예측 결과를 1차원으로 펼칠 때는 `reshape(-1)` 또는 `flatten()` 을 사용합니다.

1차원 1차원

5. 인덱싱(Indexing) 추출

비연속

여러 개

ndarray에서 원하는 데이터를 선택하는 4가지 방법 — **단일값 추출**, **슬라이싱**, **팬시 인덱싱**, **불린 인덱싱** — 을 차례로 학습합니다.

인덱싱 연속된

여러 개

조건에 맞는

값 추출

5.1 단일값 추출

```
# 1에서 부터 9 까지의 1차원 ndarray 생성
array1 = np.arange(start=1, stop=10)      (9,)
print('array1:', array1)
# index는 0 부터 시작하므로 array1[2]는 3번째 index 위치의 데이터 값을 의미
value = array1[2]      단일값: 인덱싱
print('value:', value)
print(type(value))
```

줄	코드	상세 설명
2	<code>array1 = np.arange(start=1, stop=10)</code>	키워드 인자로 <code>start=1, stop=10</code> 을 명시하여 1부터 9까지 의 배열을 생성합니다. <code>stop</code> 값 자체는 포함되지 않습니다.
5	<code>value = array1[2]</code>	대괄호 안에 인덱스 2를 지정합니다. 인덱스는 0부터 시작 하므로 세 번째 원소인 3이 추출됩니다.
6~7	<code>print(value / type(value))</code>	값 3과 타입 <code><class 'numpy.int32'></code> 가 출력됩니다. 단일값 추출 결과는 더 이상 ndarray가 아닌 NumPy 스칼라 값 입니다.

▶ 실행 결과

```
array1: [1 2 3 4 5 6 7 8 9]
value: 3
<class 'numpy.int32'>
```

```
print('맨 뒤의 값:', array1[-1], ', 맨 뒤에서 두번째 값:', array1[-2])
```

줄	코드	상세 설명
1	<code>array1[-1], array1[-2]</code>	음수 인덱스 는 뒤에서부터 셉니다. -1은 맨 마지막(9), -2는 뒤에서 두 번째(8)를 의미합니다. 배열 길이를 몰라도 끝 원소에 접근할 수 있어 편리합니다.

▶ 실행 결과

맨 뒤의 값: 9 , 맨 뒤에서 두번째 값: 8

```
array1[0] = 9
array1[8] = 0
print('array1:',array1)
```

줄	코드	상세 설명
1	array1[0] = 9	인덱싱은 값 조회뿐 아니라 수정 에도 사용됩니다. 첫 번째 원소(1)를 9로 바꿉니다.
2	array1[8] = 0	마지막(9번째) 원소를 0으로 바꿉니다.
3	print('array1:',array1)	원본 배열이 직접 변경되었음을 확인합니다.

▶ 실행 결과

array1: [9 2 3 4 5 6 7 8 0]

```
array1d = np.arange(start=1, stop=10)
array2d = array1d.reshape(3,3)
print(array2d)

print('(row=0,col=0) index 가리키는 값:', array2d[0,0] )
print('(row=0,col=1) index 가리키는 값:', array2d[0,1] )
print('(row=1,col=0) index 가리키는 값:', array2d[1,0] )
print('(row=2,col=2) index 가리키는 값:', array2d[2,2] )
```

줄	코드	상세 설명
1~2	array2d = array1d.reshape(3,3)	1~9의 1차원 배열을 3×3 2차원 배열로 변환합니다.
5	array2d[0,0]	2차원 배열은 [행 인덱스, 열 인덱스] 형태로 접근합니다. (0,0)은 첫 행 첫 열 → 1.
6	array2d[0,1]	첫 행 두 번째 열 → 2.
7	array2d[1,0]	두 번째 행 첫 열 → 4.
8	array2d[2,2]	세 번째 행 세 번째 열(우하단 모서리) → 9.

▶ 실행 결과

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
(row=0,col=0) index 가리키는 값: 1
(row=0,col=1) index 가리키는 값: 2
(row=1,col=0) index 가리키는 값: 4
(row=2,col=2) index 가리키는 값: 9
```

5.2 슬라이싱(Slicing)

시작:끝 기호로 **연속된 범위**의 데이터를 추출합니다. 시작 인덱스는 포함, 끝 인덱스는 **포함되지 않는** 점에 유의합니다.

```
array1 = np.arange(start=1, stop=10)
array3 = array1[0:3]
print(array3)
print(type(array3))
```

줄	코드	상세 설명
2	array3 = array1[0:3]	인덱스 0부터 3 직전(2) 까지, 즉 첫 3개 원소 [1 2 3]을 추출합니다.
4	print(type(array3))	단일값 추출과 달리 슬라이싱 결과는 ndarray 타입이 유지 됩니다.

▶ 실행 결과

```
[1 2 3]
<class 'numpy.ndarray'>
```

```
array1 = np.arange(start=1, stop=10)
array4 = array1[:3]
print(array4)

array5 = array1[3:]
print(array5)

array6 = array1[:]
print(array6)
```

줄	코드	상세 설명
2	array4 = array1[:3]	시작 인덱스 생략 시 처음(0) 부터로 간주 → [1 2 3].
5	array5 = array1[3:]	끝 인덱스 생략 시 마지막까지 로 간주 → [4 5 6 7 8 9].
8	array6 = array1[:]	둘 다 생략하면 전체 배열 이 선택됩니다.

▶ 실행 결과

```
[1 2 3]
[4 5 6 7 8 9]
[1 2 3 4 5 6 7 8 9]
```

```
array1d = np.arange(start=1, stop=10)
array2d = array1d.reshape(3,3)
print('array2d:\n',array2d)

print('array2d[0:2, 0:2] \n', array2d[0:2, 0:2])
print('array2d[1:3, 0:3] \n', array2d[1:3, 0:3])
print('array2d[1:3, :] \n', array2d[1:3, :])
print('array2d[:, :] \n', array2d[:, :])
print('array2d[:2, 1:] \n', array2d[:2, 1:])
print('array2d[:2, 0] \n', array2d[:2, 0])
```

줄	코드	상세 설명
5	array2d[0:2, 0:2]	행 0~1, 열 0~1 → 좌상단 2×2 부분 행렬 [[1,2],[4,5]].
6	array2d[1:3, 0:3]	행 1~2, 열 전체 → 아래 두 행 [[4,5,6],[7,8,9]].
7	array2d[1:3, :]	열에 :만 쓰면 전체 열 선택. 결과는 위와 동일합니다.

8	<code>array2d[:, :]</code>	행·열 모두 전체 → 원본과 동일한 3×3 행렬.
9	<code>array2d[:2, 1:]</code>	행 0~1, 열 1~끝 → 우상단 2×2 <code>[[2,3],[5,6]]</code> .
10	<code>array2d[:2, 0]</code>	행은 슬라이싱(<code>:</code> 2), 열은 단일 인덱스(0) → 열 축이 사라져 1차원 배열 <code>[1 4]</code> 가 반환됩니다. 슬라이싱과 단일 인덱스의 혼용 시 차원 축소에 주의합니다.

▶ 실행 결과

```
array2d:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
array2d[0:2, 0:2]
[[1 2]
 [4 5]]
array2d[1:3, 0:3]
[[4 5 6]
 [7 8 9]]
array2d[1:3, :]
[[4 5 6]
 [7 8 9]]
array2d[:, :]
[[1 2 3]
 [4 5 6]
 [7 8 9]]
array2d[:2, 1:]
[[2 3]
 [5 6]]
array2d[:2, 0]
[1 4]
```

```
print(array2d[0])
print(array2d[1])
print('array2d[0] shape:', array2d[0].shape, 'array2d[1] shape:', array2d[1].shape )
```

줄	코드	상세 설명
1~2	<code>array2d[0], array2d[1]</code>	2차원 배열에 행 인덱스 하나만 지정하면 해당 행 전체가 1차원 배열 로 반환됩니다. <code>array2d[0] → [1 2 3]</code> .
3	<code>print(... shape ...)</code>	반환된 배열의 shape가 (3,)임을 확인 → 2차원에서 한 차원이 줄어든 결과입니다.

▶ 실행 결과

```
[1 2 3]
[4 5 6]
array2d[0] shape: (3,) array2d[1] shape: (3,)
```

5.3 팬시 인덱싱(Fancy Indexing)

인덱스 위치에 **리스트나 ndarray를 전달**하여, 연속이 아닌 **임의의 위치들**을 한꺼번에 선택하는 방식입니다.

```
array1d = np.arange(start=1, stop=10)
array2d = array1d.reshape(3,3)
```

```
array3 = array2d[[0,1], 2]
print('array2d[[0,1], 2] => ',array3.tolist())

array4 = array2d[[0,1], 0:2]
print('array2d[[0,1], 0:2] => ',array4.tolist())

array5 = array2d[[0,1]]
print('array2d[[0,1]] => ',array5.tolist())
```

줄	코드	상세 설명
4	array3 = array2d[[0,1], 2]	행은 리스트 [0,1], 열은 단일 인덱스 2 → (0,2)와 (1,2) 위치의 값 [3, 6]이 1차원으로 반환됩니다.
7	array4 = array2d[[0,1], 0:2]	행은 [0,1], 열은 슬라이싱 0:2 → 0행의 [1,2]와 1행의 [4,5]로 구성된 2차원 [[1,2],[4,5]]가 반환됩니다.
10	array5 = array2d[[0,1]]	열 인덱스를 생략하면 지정한 행 전체 가 선택되어 [[1,2,3],[4,5,6]]이 반환됩니다.

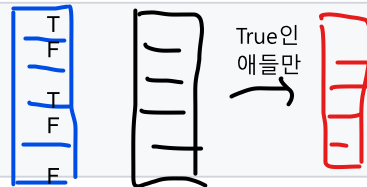
▶ 실행 결과

```
array2d[[0,1], 2] => [3, 6]
array2d[[0,1], 0:2] => [[1, 2], [4, 5]]
array2d[[0,1]] => [[1, 2, 3], [4, 5, 6]]
```

5.4 불린 인덱싱(Boolean Indexing)

조건식을 [] 안에 직접 기술하여 **조건을 만족하는 데이터만 추출**하는, 실무에서 가장 자주 사용되는 인덱싱 방식입니다.

```
array1d = np.arange(start=1, stop=10)
# [ ] 안에 array1d > 5 Boolean indexing을 적용
array3 = array1d[array1d > 5] Series
print('array1d > 5 불린 인덱싱 결과 값 : ', array3)
```



줄	코드	상세 설명
3	<pre>array3 = array1d[조건 array1d > 5] shape (9,) 브로드캐스팅 (3,3) 5 5 5 5 5 5 5 5 5 < < < < < < < < <-- 원소별 연산(벡터 연산) : 병렬 처리</pre>	내부적으로 2단계로 동작합니다. ① array1d > 5가 각 원소별 True/False로 이뤄진 불린 배열 을 생성하고, ② 그 불린 배열을 인덱스로 사용해 True 위치의 값만 반환합니다. for 루프와 if 조건문 없이 한 줄로 필터링이 완성됩니다.

▶ 실행 결과

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
F F F F F T T T T
```

```
array1d > 5 불린 인덱싱 결과 값 : [6 7 8 9]
```

```
array1d > 5
```

줄	코드	상세 설명
1	array1d > 5	비교 연산자가 배열 전체에 원소 단위(element-wise) 로 적용되어 같은 크기의 불린 배열이 반환됩니다. 1~5는 False, 6~9는 True.

▶ 실행 결과

```
array([False, False, False, False, False, True, True, True, True])
```

```
boolean_indexes = np.array([False, False, False, False, False, True, True, True, True])
array3 = array1d[boolean_indexes]
print('불린 인덱스로 필터링 결과 :', array3)
```

줄	코드	상세 설명
1	<code>boolean_indexes = np.array([False, ..., True])</code>	위 조건식의 결과와 동일한 불린 배열을 수동으로 만들어 줍니다.
2	<code>array3 = array1d[boolean_indexes]</code>	이 불린 배열을 인덱스로 넣으면 True인 위치(인덱스 5~8)의 값 [6 7 8 9]만 추출됩니다. 조건식 방식과 결과가 같음을 확인 → 불린 인덱싱의 내부 동작 원리를 이해할 수 있습니다.

▶ 실행 결과

불린 인덱스로 필터링 결과 : [6 7 8 9]

```
indexes = np.array([5,6,7,8])
array4 = array1d[ indexes ]
print('일반 인덱스로 필터링 결과 :',array4)
```

줄	코드	상세 설명
1~3	<code>indexes = np.array([5,6,7,8]); array1d[indexes]</code>	동일한 결과를 일반(팬시) 인덱스 로 얻으려면 원하는 위치 번호를 직접 나열해야 합니다. 결과는 같지만, 조건이 복잡해질수록 위치를 일일이 계산해야 하므로 불린 인덱싱이 훨씬 효율적입니다.

▶ 실행 결과

일반 인덱스로 필터링 결과 : [6 7 8 9]

■ 머신러닝 실무 포인트

불린 인덱싱은 이상치 제거($X[X < \text{threshold}]$), 특정 클래스 샘플 선택($X[y == 1]$) 등 데이터 전처리 전반에서 핵심적으로 사용됩니다. 복수 조건은 `&` (and), `|` (or)와 괄호를 함께 사용합니다. 예: `array1d[(array1d > 3) & (array1d < 8)]`

ndarray

6. 행렬의 정렬 - sort()와 argsort()

6.1 **np.sort()** vs **ndarray.sort()**

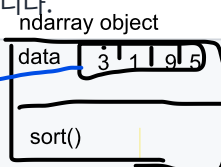
모듈 → 함수 → 정렬할 값 넣어야 → 변수

object 메소드 메소드

data
sort

NumPy의 정렬에는 두 가지 방식이 있으며, **원본 보존 여부**가 결정적인 차이입니다.
함수와 메소드의 차이.

```
원본 ndarray
org_array = np.array([ 3, 1, 9, 5])
print('원본 행렬:', org_array)
# np.sort()로 정렬
sort_array1 = np.sort(org_array)
print('np.sort() 호출 후 반환된 정렬 행렬:', sort_array1) [1,3,5,9]
print('np.sort() 호출 후 원본 행렬:', org_array) [3,1,5,9]
# ndarray.sort()로 정렬
sort_array2 = org_array.sort()
# ndarray.sort() 메소드(원래 data 정렬. 원본 변형)
```



함수
메소드 return해야 한다.


```
print('org_array.sort( ) 호출 후 반환된 행렬:', sort_array2)
print('org_array.sort( ) 호출 후 원본 행렬:', org_array)
```

줄	코드	상세 설명
1~2	org_array = np.array([3, 1, 9, 5])	정렬되지 않은 4개 원소의 배열을 생성하고 원본을 출력합니다.
4	sort_array1 = np.sort(org_array)	np.sort() 함수 방식: 원본은 그대로 두고 정렬된 새 배열을 반환 합니다.
5~6	print(...)	반환값은 [1 3 5 9]로 정렬됐지만, 원본은 [3 1 9 5] 그대로 유지 됩니다.
8	sort_array2 = org_array.sort()	ndarray.sort() 메서드 방식: 원본 자체를 정렬(in-place) 하고 반환값은 None 입니다.
9~10	print(...)	반환값은 None, 원본은 [1 3 5 9]로 변경 되었습니다. 반환값을 변수에 담아 쓰려다 None이 나오는 실수가 매우 흔하므로 주의합니다.

▶ 실행 결과

```
원본 행렬: [3 1 9 5]
np.sort( ) 호출 후 반환된 정렬 행렬: [1 3 5 9]
np.sort( ) 호출 후 원본 행렬: [3 1 9 5]
org_array.sort( ) 호출 후 반환된 행렬: None
org_array.sort( ) 호출 후 원본 행렬: [1 3 5 9]
```

6.2 내림차순 정렬

```
sort_array1_desc = np.sort(org_array)[::-1]
print('내림차순으로 정렬:', sort_array1_desc)
```

줄	코드	상세 설명
1	np.sort(org_array)[::-1]	NumPy에는 내림차순 옵션이 따로 없어, 오름차순 정렬 후 [::-1] 슬라이싱으로 역순 배치 하는 관용 표현을 사용합니다. [시작:끝:간격]에서 간격 -1은 "뒤에서부터 거꾸로"를 의미합니다.

▶ 실행 결과

```
내림차순으로 정렬: [9 5 3 1]
```

6.3 2차원 배열의 축(axis) 방향 정렬

```
array2d = np.array([[8, 12],
                    [7, 1 ]])

sort_array2d_axis0 = np.sort(array2d, axis=0)
print('로우 방향으로 정렬:\n', sort_array2d_axis0)

sort_array2d_axis1 = np.sort(array2d, axis=1)
print('컬럼 방향으로 정렬:\n', sort_array2d_axis1)
```

줄	코드	상세 설명
1~2	array2d = np.array([[8,12],[7,1]])	2×2 2차원 배열을 생성합니다.

4	<code>np.sort(array2d, axis=0)</code>	axis=0(로우 방향) : 같은 열 안에서 세로로 비교·정렬합니다. 1열: 8과 7 → 7,8 / 2열: 12와 1 → 1,12. 결과 <code>[[7,1],[8,12]]</code> .
7	<code>np.sort(array2d, axis=1)</code>	axis=1(컬럼 방향) : 같은 행 안에서 가로로 정렬합니다. 1행: 8,12 그대로 / 2행: 7,1 → 1,7. 결과 <code>[[8,12],[1,7]]</code> .

▶ 실행 결과

```
로우 방향으로 정렬:
[[ 7  1]
 [ 8 12]]
컬럼 방향으로 정렬:
[[ 8 12]
 [ 1  7]]
```

6.4 argsort() - 정렬 행렬의 인덱스 반환

`np.argsort( )` 는 정렬된 값이 아니라 "정렬됐을 때 원본 요소들이 있던 인덱스"를 반환합니다.

```
org_array = np.array([ 3, 1, 9, 5])
sort_indices = np.argsort(org_array)
print(type(sort_indices))
print('행렬 정렬 시 원본 행렬의 인덱스:', sort_indices)
```

줄	코드	상세 설명
2	<code>sort_indices = np.argsort(org_array)</code>	[3,1,9,5]를 오름차순 정렬하면 [1,3,5,9]인데, 이 값들의 원본 위치 는 1(값 1), 0(값3), 3(값5), 2(값9)입니다. 따라서 [1 0 3 2]가 반환됩니다.
3~4	<code>print(type / 값)</code>	반환 타입도 ndarray이며, 값은 [1 0 3 2]입니다.

▶ 실행 결과

```
<class 'numpy.ndarray'>
행렬 정렬 시 원본 행렬의 인덱스: [1 0 3 2]
```

```
org_array = np.array([ 3, 1, 9, 5])
sort_indices_desc = np.argsort(org_array)[::-1]
print('행렬 내림차순 정렬 시 원본 행렬의 인덱스:', sort_indices_desc)
```

줄	코드	상세 설명
2	<code>np.argsort(org_array)[::-1]</code>	argsort 결과에 <code>[::-1]</code> 을 적용해 내림차순 기준의 인덱스 [2 3 0 1]을 얻습니다. (9→인덱스2, 5→3, 3→0, 1→1)

▶ 실행 결과

```
행렬 내림차순 정렬 시 원본 행렬의 인덱스: [2 3 0 1]
```

6.5 argsort() 활용 - 다른 배열을 기준으로 정렬하기

```
import numpy as np

name_array = np.array(['John', 'Mike', 'Sarah', 'Kate', 'Samuel'])
score_array = np.array([78, 95, 84, 98, 88])
```

```
sort_indices_asc = np.argsort(score_array)
print('성적 오름차순 정렬 시 score_array의 인덱스:', sort_indices_asc)
print('성적 오름차순으로 name_array의 이름 출력:', name_array[sort_indices_asc])
```

줄	코드	상세 설명
3~4	name_array / score_array	학생 이름 배열과 성적 배열을 같은 순서로 대응 되도록 생성합니다. (John-78, Mike-95, ...)
6	sort_indices_asc = np.argsort(score_array)	성적을 오름차순 정렬할 때의 인덱스를 구합니다. 78(0) → 84(2) → 88(4) → 95(1) → 98(3)이므로 [0 2 4 1 3].
8	name_array[sort_indices_asc]	이 인덱스를 name_array에 팬시 인덱싱 으로 적용하면 성적 순서대로 이름 이 재배열됩니다. ndarray는 별도의 키-값 구조가 없으므로 argsort가 이런 "기준 열 정렬" 역할을 대신합니다.

▶ 실행 결과

```
성적 오름차순 정렬 시 score_array의 인덱스: [0 2 4 1 3]
성적 오름차순으로 name_array의 이름 출력: ['John' 'Sarah' 'Samuel' 'Mike' 'Kate']
```

■ 머신러닝 실무 포인트

argsort() 는 모델의 **피쳐 중요도(feature importance)** 상위 N개 추출, 예측 확률 상위 후보 선택 등에서 **매우 자주 사용**됩니다. 예: `np.argsort(model.feature_importances_)[:-1][:5]` → 중요도 상위 5개 피쳐의 인덱스.

7. 선형대수 연산 - 행렬 내적과 전치 행렬

관계값 : 얼마나 관계되어 있는지. 값이 클 수록 관계가 깊다.

7.1 행렬 내적(행렬 곱) - np.dot() ★ 정보(데이터)의 유사도, 영향력

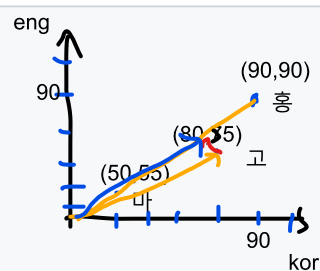
행렬 내적은 **왼쪽 행렬의 행(row)**과 **오른쪽 행렬의 열(column)**을 원소끼리 곱해 더한 값으로 새 행렬을 만드는 연산입니다. 왼쪽 행렬의 열 수와 오른쪽 행렬의 행 수가 같아야 합니다.

```
A = np.array([[1, 2, 3],
               [4, 5, 6]])
```

```
B = np.array([[7, 8],
               [9, 10],
               [11, 12]])
```

```
dot_product = np.dot(A, B)
print('행렬 내적 결과:\n', dot_product)
```

축
컬럼 축
 컬럼
kor eng
홍 90 90
고 80 75
마 50 55



내적. 얼마나 유사한지.(유사도) 상관계수.

줄	코드	상세 설명
1~2	A = np.array([[1,2,3],[4,5,6]])	2×3 크기의 행렬 A를 생성합니다.
3~5	B = np.array([[7,8],[9,10],[11,12]])	3×2 크기의 행렬 B를 생성합니다. A의 열 수(3) = B의 행 수(3)이므로 내적이 가능하며, 결과는 (2×3)·(3×2) = 2×2 행렬이 됩니다.

7	<code>dot_product = np.dot(A, B)</code>	내적을 계산합니다. 각 원소의 계산 과정: (1,1) = $1 \times 7 + 2 \times 9 + 3 \times 11 = 7 + 18 + 33 = 58$ (1,2) = $1 \times 8 + 2 \times 10 + 3 \times 12 = 8 + 20 + 36 = 64$ (2,1) = $4 \times 7 + 5 \times 9 + 6 \times 11 = 28 + 45 + 66 = 139$ (2,2) = $4 \times 8 + 5 \times 10 + 6 \times 12 = 32 + 50 + 72 = 154$ 동일한 연산을 <code>A @ B</code> 연산자로도 수행할 수 있습니다.
---	---	--

▶ 실행 결과

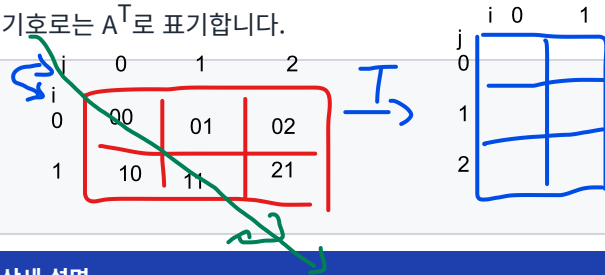
행렬 내적 결과:

```
[[ 58  64]
 [139 154]]
```

7.2 전치 행렬 - `np.transpose()`

전치 행렬은 원 행렬의 행과 열을 서로 맞바꾼 행렬로, 수학 기호로는 A^T 로 표기합니다.

```
A = np.array([[1, 2],
              [3, 4]])
transpose_mat = np.transpose(A)
print('A의 전치 행렬:\n', transpose_mat)
```



줄	코드	상세 설명
1~2	<code>A = np.array([[1,2],[3,4]])</code>	2×2 행렬 A를 생성합니다.
3	<code>transpose_mat = np.transpose(A)</code>	(i, j) 위치의 원소가 (j, i)로 이동합니다. 2(0행1열) ↔ 3(1행0열)이 서로 자리를 바꿉니다. $A.T$ 속성으로 더 간단히 쓸 수도 있습니다.

▶ 실행 결과

A의 전치 행렬:

```
[[1 3]
 [2 4]]
```

■ 머신러닝 실무 포인트

행렬 내적은 선형 회귀의 예측값 계산 ($X @ w$), 신경망의 층 간 연산 등 머신러닝 수학의 중심 연산입니다. 전치는 데이터의 행/열 방향을 맞추거나 정규방정식 ($(X^T X)^{-1} X^T y$) 계산 등에 사용됩니다.

8. 핵심 요약 및 머신러닝 활용 포인트

8.1 이번 장 핵심 API 요약표

구분	API	기능	주의 사항
생성	<code>np.array(리스트)</code>	리스트 → ndarray 변환	혼합 타입은 큰 타입으로 일괄 변환
생성	<code>np.arange(n)</code>	0 ~ n-1 연속 값 배열	stop 값 미포함
생성	<code>np.zeros</code> / <code>np.ones</code>	0 / 1로 초기화된 배열	dtype 미지정 시 float64
속성	<code>shape</code> / <code>ndim</code> / <code>dtype</code>	크기 / 차원 수 / 데이터 타입	(3,)은 1차원, (1,3)은 2차원

변환	<code>astype('타입')</code>	명시적 형 변환	float→int는 소수점 버림
변환	<code>reshape(행, 열)</code>	차원·크기 변경	원소 수 불일치 시 ValueError, -1은 자동 계산
인덱싱	<code>arr[i], arr[i,j]</code>	단일값 추출	결과는 스칼라(차원 축소)
인덱싱	<code>arr[a:b]</code>	슬라이싱	끝 인덱스 미포함, 결과는 ndarray
인덱싱	<code>arr[[0,2], :]</code>	팬시 인덱싱	임의 위치를 리스트로 지정
인덱싱	<code>arr[조건식]</code>	불린 인덱싱	복수 조건은 &, 와 괄호 사용
정렬	<code>np.sort(arr)</code>	정렬된 새 배열 반환	원본 유지
정렬	<code>arr.sort()</code>	원본 자체를 정렬	반환값은 None
정렬	<code>np.argsort(arr)</code>	정렬 시 원본 인덱스 반환	다른 배열 재정렬에 활용
선형대수	<code>np.dot(A, B)</code>	행렬 내적	A 열 수 = B 행 수 필요, @ 연산자 동일
선형대수	<code>np.transpose(A)</code>	전치 행렬	<code>A.T</code> 와 동일

8.2 사이킷런 학습을 위한 연결 고리

- **2차원 피쳐 행렬(X)** : 사이킷런의 모든 학습 데이터는 (샘플 수, 피쳐 수) 형태의 2차원 ndarray입니다. shape와 reshape를 자유롭게 다뤄야 데이터 준비가 가능합니다.
- **불린 인덱싱** : 결측/이상치 필터링, 클래스별 샘플 분리 등 전처리의 핵심 도구입니다.
- **argsort** : 피쳐 중요도 순위, 예측 확률 상위 추출 등 모델 해석 단계에서 반복적으로 등장합니다.
- **dtype 관리** : 문자열이 섞여 object/유니코드 타입이 되면 모델 학습이 불가능하므로, 인코딩 전 dtype 확인이 필수입니다.
- **행렬 내적·전치** : 선형 회귀, PCA, 신경망 등 대부분의 알고리즘 수식이 이 두 연산으로 표현됩니다.

⚠ 안정화 버전 참고

본 자료의 모든 코드는 NumPy 1.24 이상의 안정화 버전에서 동작을 확인했습니다. NumPy 2.x에서도 본 장의 API는 모두 동일하게 동작하지만, 정수 기본 dtype 표기(int32/int64)는 운영체제(Windows/Linux)에 따라 다르게 출력될 수 있습니다.